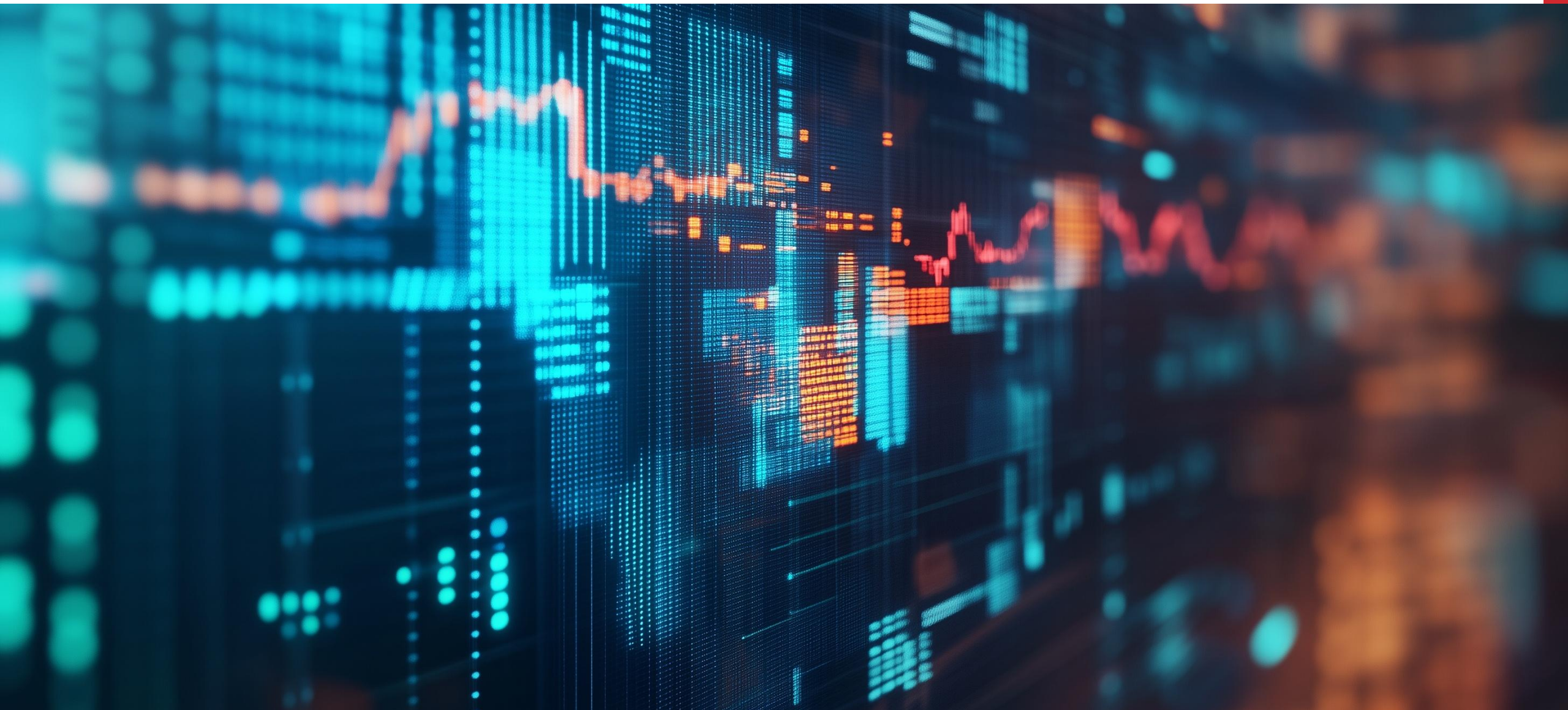
Modern Tooling and DevOps Integration



WHAT IS A REST API?

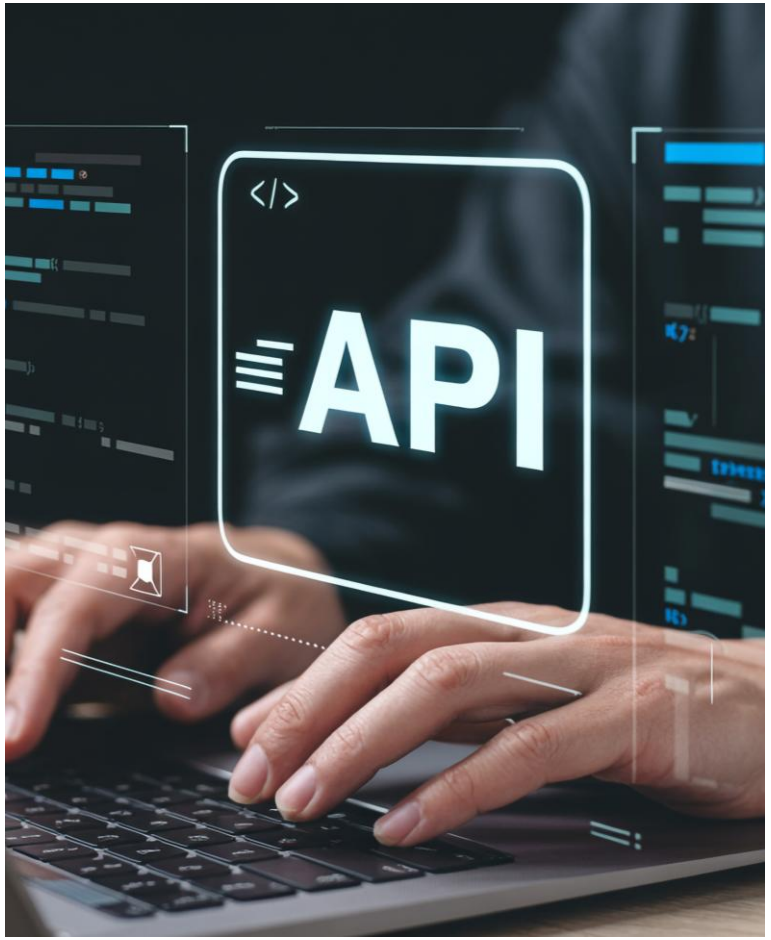
A REST API is a way for software systems to communicate over HTTP using standardized rules. It exposes data and actions through URLs and HTTP methods rather than user interfaces.

- Designed for program-to-program communication
- Uses HTTP as the transport protocol
- Common in web services and automation tooling

REST IS RESOURCE-ORIENTED

REST APIs organize data around resources instead of actions or commands. Each resource is identified by a URL and manipulated using HTTP methods.

- Resources are nouns like /users or /routes
- URLs identify what data is being accessed
- HTTP methods define the action taken



STATELESS COMMUNICATION

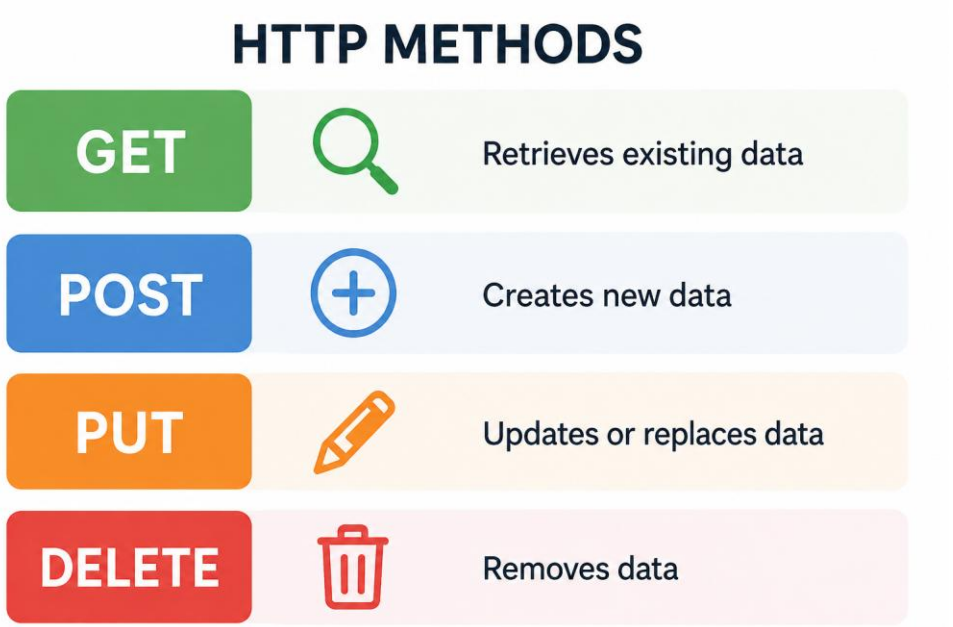
REST APIs are stateless, meaning the server does not remember previous requests. Every request must include all information needed to process it.

- No server-side session memory by default
- Improves scalability and reliability
- Clients are responsible for context

HTTP METHODS IN PRACTICE

HTTP methods describe what action a client wants to perform on a resource. Using the correct method is critical for predictable API behavior.

- GET retrieves existing data
- POST creates new data
- PUT updates or replaces data
- DELETE removes data



WHY REST MATTERS FOR AUTOMATION

Most modern infrastructure and cloud platforms expose REST APIs. Automation scripts rely on REST to create, modify, and inspect systems programmatically.

- Used by cloud providers, network devices, and SaaS platforms
- Enables repeatable and scriptable workflows
- Forms the foundation of API-driven automation

REST Communication

All REST communication happens through HTTP requests and responses.
The client sends a request and the server returns a structured response.

REQUESTS INCLUDE

- Method (GET, POST, PUT, DELETE)
- URL
- Headers
- Optional Body

RESPONSES INCLUDE

- Status Code (e.g., 200, 404)
- Headers
- Optional Body

Clients must interpret responses correctly.

HTTP Headers

Headers provide metadata about an HTTP request.
They tell the server how to interpret the request and its contents.

Content-Type

describes the request body format

Accept

tells the server what response formats are allowed

Headers

are key-value pairs

Example

```
Content-Type: application/json
Accept: application/json
Authorization: Bearer <token>
```

RESPONSE HEADERS

Response headers provide metadata about the server's reply. They help clients understand how to process the response.

- Content-Type describes the response body format
- Headers can control caching and behavior
- Clients rely on headers for parsing

HTTP STATUS CODES

STATUS CODE GROUP	MEANING	COMMON EXAMPLES	WHAT IT USUALLY INDICATES
2xx Success	The request was received, understood, and successfully processed.	• 200 OK • 201 Created • 204 No Content	Everything worked as expected.
3xx Redirection	Additional action is required to complete the request, usually a redirect.	• 301 Moved Permanently • 302 Found • 304 Not Modified	The resource has moved or the client should use cached content.
4xx Client Error	The request contains invalid data or the client is not authorized to perform the action.	• 400 Bad Request • 401 Unauthorized • 403 Forbidden • 404 Not Found	The client must correct something before retrying.
5xx Server Error	The server encountered an unexpected condition while processing the request.	• 500 Internal Server Error • 502 Bad Gateway • 503 Service Unavailable • 504 Gateway Timeout	The problem exists on the server side, not the client side.

JSON in REST APIs

JSON is the most common data format used in REST APIs.
It balances readability for humans with ease of parsing for machines.

Lightweight
text-based format

Maps Cleanly
to Python dictionaries

Widely Supported
across platforms

JSON Example

```
{
   "id": 101,
   "name": "Alice",
   "email": "alice@example.com",
   "active": true,
   "roles": ["user", "admin"],
   "profile": {
      "age": 28,
      "city": "New York"
   }
}
```

Python Dictionary Equivalent

```
{
   'id': 101,
   'name': 'Alice',
   'email': 'alice@example.com',
   'active': True,
   'roles': ['user', 'admin'],
   'profile': {
      'age': 28,
      'city': "New York"
   }
}
```

JSON's simplicity and flexibility make it the ideal choice for data exchange in REST APIs.

APIS ARE CONTRACTS

An API defines a contract between a client and a server. Breaking that contract causes client applications to fail.

- Endpoints should remain stable
- Response formats should be consistent
- Changes require coordination

REST API Versioning

Versioning allows you to evolve your API without breaking existing clients.

COMMON VERSIONING SCHEMAS

1 URI Path (Most Common)

Include the version in the URL path.

```
https://api.example.com/v1/users
```

2 Query Parameter

Pass the version as a query parameter.

```
https://api.example.com/users?version=1
```

3 Header

Specify the version in a custom request header.

```
X-API-Version: 1
```

4 Media Type (Accept Header)

Use media type parameters to indicate version.

```
Accept: application/vnd.api+json; version=1
```

WHY VERSION?

- Introduce new features and improvements
- Modify or remove existing fields
- Maintain backward compatibility
- Give clients time to migrate

/v1/users/{id} (Version 1)

Response Schema (Example)

```
{
  "id": 101,
  "name": "Alice",
  "email": "alice@example.com",
  "role": "user"
}
```

Characteristics

- Basic user information
- "role" field as a string

What Changed in v2?

- Added "createdAt" (field added)
- Changed "role" (string → array)
- Removed "email" (field removed)

/v2/users/{id} (Version 2)

Response Schema (Example)

```
{
  "id": 101,
  "name": "Alice",
  "role": ["user", "admin"],
  "createdAt": "2024-05-01T10:30:00Z"
}
```

Characteristics

- "email" field removed
- "role" is now an array (more flexible)
- "createdAt" timestamp added

Handling Resource Identifiers in URLs

REST APIs often encode resource identifiers in the URL.
Handlers must safely extract and validate these identifiers.

Modern (Preferred)

Client

GET

/users/123

Path Parameter

ID is part of the path.

`/users/123`

Clear, RESTful, cache-friendly, easy to route.

Old School (Query Parameter)

Client

GET

/users?id=123

Query Parameter

ID is passed as a query string.

`/users?id=123`

Common in older APIs, less RESTful.

Server (Handler)

1 Extract ID

Path: split by '/'
Query: read 'id' parameter

2 Convert to Integer

Convert ID to integer carefully (e.g., int())

3 Validate

Check if ID is valid and resource exists

Valid → **200 OK** Return resource

Invalid / Not Found → **400 Bad Request or 404 Not Found**

Examples

Request	When to Use	Example	Notes
Path Parameter	Most RESTful way to identify a specific resource	`GET /users/123`	Preferred for new APIs
Query Parameter	Useful for search, filtering, or legacy APIs	`GET /users?id=123`	Still widely used

Validation Rules

- Split paths using '/'
- Convert IDs to integers carefully
- Invalid format → 400 Bad Request
- ID not found → 404 Not Found

POST vs PUT: Data in the Body, Not in the URL

In REST APIs, data is commonly sent in the request body, not in the URL.

POST (Create Resource)

POST

/users

Request Body (JSON)

```
{
  "name": "Alice",
  "email": "alice@example.com",
  "role": "user"
}
```

Server creates the resource.
Returns 201 Created + location of new resource.

Don't do this

Avoid sending data in the URL:

```
POST /users?name=Alice&email=alice@example.com
```

- Exposes data
- Length limits
- Not secure

Response (201 Created)

```
Location: /users/123
{
  "id": 123,
  "name": "Alice",
  "email": "alice@example.com",
  "role": "user"
}
```

PUT (Update or Replace Resource)

PUT

/users/123

Request Body (JSON)

```
{
  "name": "Alice Smith",
  "email": "alice@example.com",
  "role": "admin"
}
```

Server updates or replaces the resource with ID 123.
Returns 200 OK (or 204 No Content).

Don't do this

Avoid sending data in the URL:

```
PUT /users/123?name=Alice Smith&role=admin
```

- Exposes data
- Length limits
- Not secure

Response (200 OK)

```
{
  "id": 123,
  "name": "Alice Smith",
  "email": "alice@example.com",
  "role": "admin"
}
```

Best Practices

- Send data in the request body, not in the URL.
- Keeps data secure and out of logs, bookmarks, and browser history.
- Use **POST** to create new resources.
- Use **PUT** to update or replace an existing resource.
- Use appropriate status codes: 201 Created, 200 OK, or 204 No Content.

Parsing JSON Request Bodies in REST APIs

JSON request bodies must be parsed into Python objects.
Invalid JSON should always be treated as a client error.

REQUEST FLOW

BEST PRACTICES

- Use `json.loads()` to parse JSON.
- Wrap parsing in `try/except` to catch errors.
- Return 400 Bad Request on malformed JSON.

PYTHON HANDLER (SERVER)

```python
import json

def handle_post_users(handler):
    content_type = handler.headers.get('Content-Type', '')
    if content_type != 'application/json':
        handler.send_response(415)  # Unsupported Media Type
        return

    try:
        data = json.loads(handler.rfile.read().decode('utf-8'))
```

Parsed into Python object (dict)

```
{'name': 'Alice', 'email': 'alice@example.com'}
```

Continue processing

HANDLING INVALID JSON

Invalid JSON Example

```
{
  "name": "Alice",
  "email": "alice@example.com",
}                    // Trailing comma
```

```python
try:
    data = json.loads(body)
except json.JSONDecodeError:
    handler.send_response(400)
    handler.send_header('Content-Type', 'application/json')
    handler.end_headers()
    handler.wfile.write(b'{"error": \"Invalid JSON"}')
    return
```

Response to Client

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{"error": "Invalid JSON"}
```

Malformed JSON is a client error.
Return 400 Bad Request.

RETURNING JSON OUTPUT

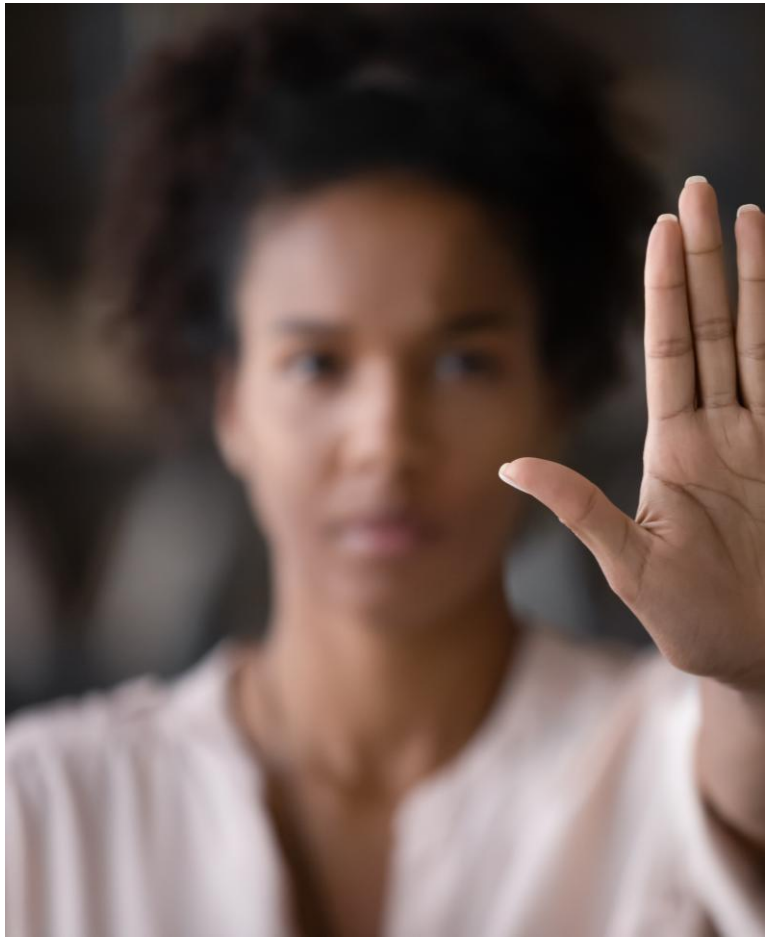
Servers return JSON by serializing Python objects. The response must include both data and correct headers.

- Use json.dumps() to serialize
- Always set Content-Type
- Encode strings to bytes before writing

PROTECTING CRITICAL RESOURCES

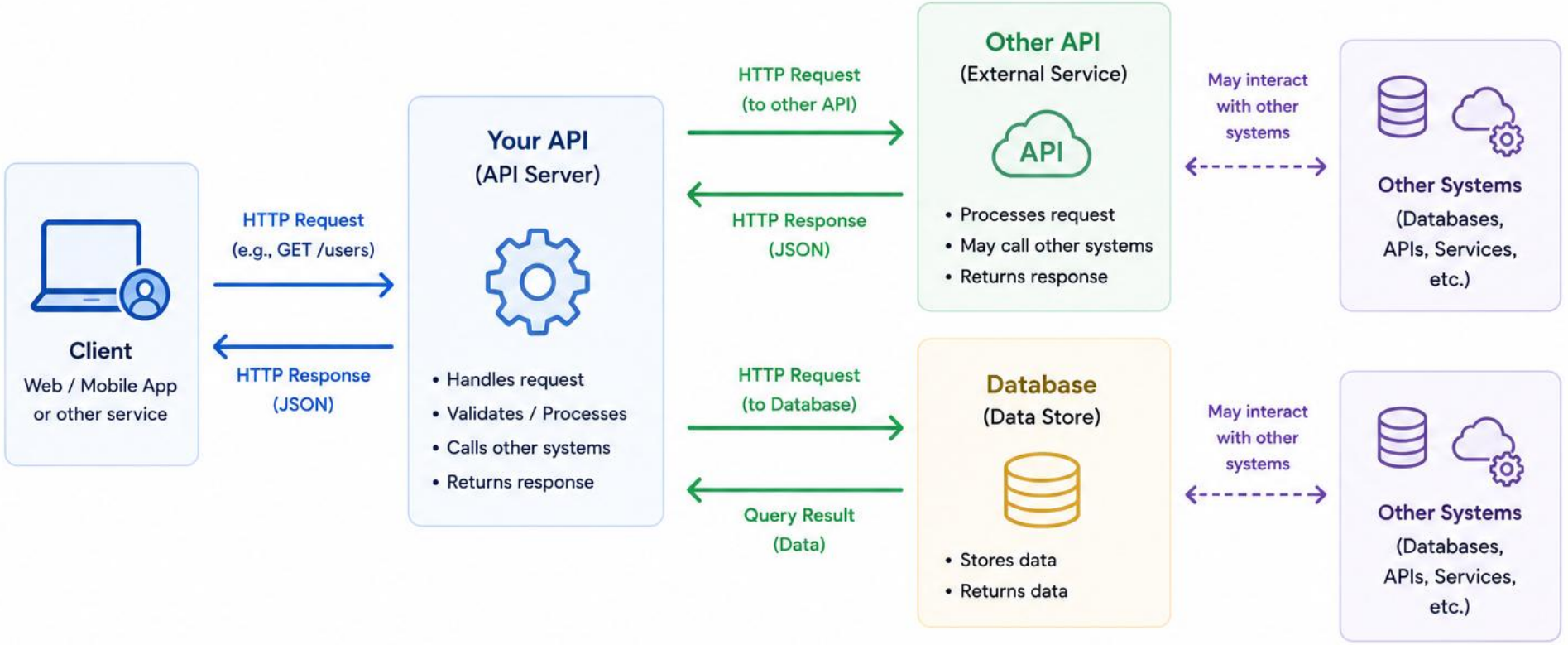
Some resources should not be modified or deleted. APIs should enforce these rules consistently.

- Check resource attributes before deletion
- Return 403 for forbidden actions
- Never rely on client behavior



APIs Work Together with Other Systems

APIs often interact with other systems such as other APIs and databases.
Those other APIs may interact with other systems as well.



Key Takeaways

- APIs rarely work in isolation.
- They often call other APIs and databases to get work done.
- Each system has a focused responsibility.
- This creates a connected ecosystem of services.

SERVER STABILITY

Uncaught exceptions can crash your API server. Handlers should protect against unexpected failures.

- Wrap logic in try/except
- Return 500 on unexpected errors
- Do not expose stack traces

TESTING WITH CURL

curl is a simple way to test HTTP APIs from the command line. It allows you to send requests exactly like a client would.

- Use -X to specify HTTP methods
- Use -H to set headers
- Use -d to send JSON bodies
- Use –c (cookie jar) to save cookie file
- Use -b to use a cookie file

```
# Login and save cookies
curl -c cookies.txt \
  -X POST https://example.com/login \
  -H "Content-Type: application/json" \
  -d '{"username":"daniel","password":"secret"}'

# Authenticated request using the saved cookie
curl -b cookies.txt \
  https://example.com/api/me
```

POP QUIZ: REST

Which HTTP method is used to update an existing resource?

A. GET

B. POST

C. PUT

D. DELETE

POP QUIZ: REST

Which HTTP method is used to update an existing resource?

A. GET

B. POST

C. PUT

D. DELETE

POP QUIZ: REST

Which status code indicates a new resource was created?

A. 200

B. 201

C. 204

D. 400

POP QUIZ: REST

Which status code indicates a new resource was created?

A. 200

B. 201

C. 204

D. 400

POP QUIZ: REST

Which header tells the server how to parse a request body?

A. Accept

B. Host

C. Content-Type

D. User-Agent

POP QUIZ: REST

Which header tells the server how to parse a request body?

A. Accept

B. Host

C. Content-Type

D. User-Agent

POP QUIZ: REST

Which request should NOT change server state?

A. POST

B. PUT

C. GET

D. DELETE

POP QUIZ: REST

Which request should NOT change server state?

A. POST

B. PUT

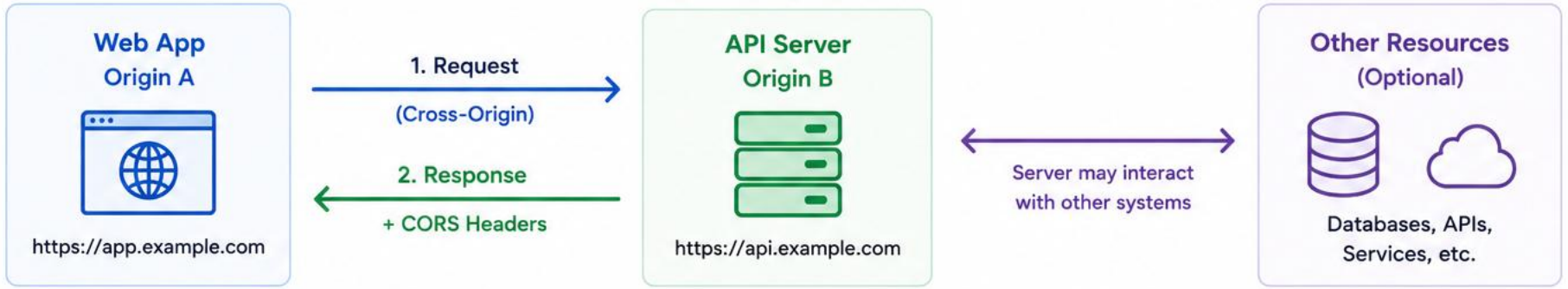
C. GET

D. DELETE

CORS (Cross-Origin Resource Sharing)

CORS is a security mechanism that allows a web application running in one origin to make requests to a different origin.

HOW IT WORKS



IMPORTANT RESPONSE HEADERS

Header	Purpose
Access-Control-Allow-Origin	Specifies which origin is allowed to access the resource
Access-Control-Allow-Methods	Specifies allowed HTTP methods (GET, POST, PUT, etc.)
Access-Control-Allow-Headers	Specifies allowed request headers
Access-Control-Allow-Credentials	Indicates if cookies/credentials are allowed

SIMPLE EXAMPLE

Request from https://app.example.com to https://api.example.com

```
GET /users HTTP/1.1
Origin: https://app.example.com
...
```

Response with CORS headers

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization

{ "data": "..." }
```

KEY POINTS

- Browsers enforce CORS. Non-browser clients (like Postman, curl) do not.
- The server must explicitly allow cross-origin access using headers.
- CORS does not replace authentication or authorization.
- Misconfiguration can expose your API to untrusted origins.

IMPLEMENTING CRUD WITH FLASK

Flask routes can be mapped directly to CRUD operations. Each HTTP method corresponds to a specific action.

- GET reads data
- POST creates data
- PUT updates data
- DELETE removes data

GET ENDPOINTS

GET routes retrieve data without changing server state. They should always be safe and repeatable.

- Typically return JSON responses
- Never modify data, Idempotent
- Use status code 2xx or 3xx if forwarding requests

POST ENDPOINTS

POST routes accept data from the client to create new resources. Servers must validate input before storing it.

- Data often sent with Body
- Usually return 201 on success
- Reject invalid input

PUT ENDPOINTS

PUT routes update existing resources identified by the URL. Clients must specify which resource to modify.

- Check resource existence
- Return updated object
- Put has a “U” for Update

DELETE ENDPOINTS

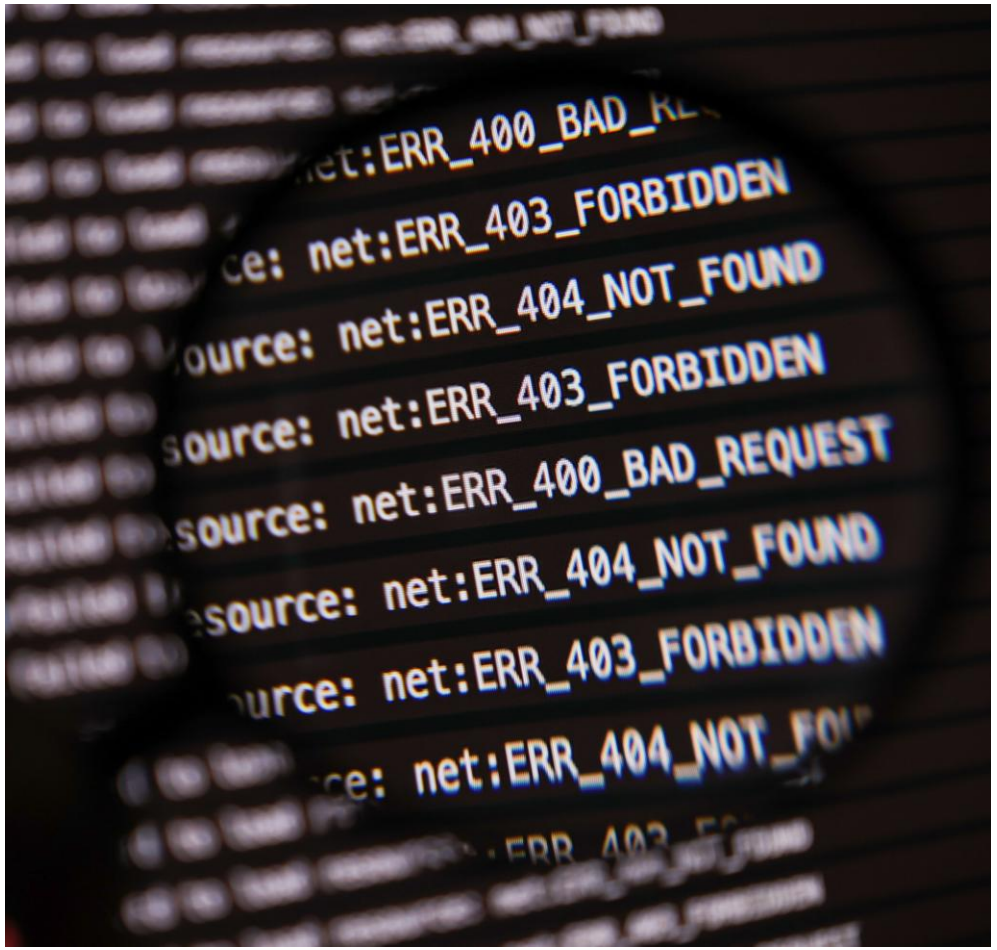
DELETE routes remove resources from the server. They should clearly indicate success or failure.

- Return 204 on success
- Return 404 if missing
- Protect critical data

LOGGING AND TELEMETRY

Logging helps track usage and diagnose issues. APIs should log key request metadata.

- Log method and path
- Log client IP and User-Agent
- Avoid logging sensitive data
- Use /metrics endpoint and OTEL for enhanced telemetry



POP QUIZ: REST

Which status code is best for a successful DELETE?

A. 200

B. 201

C. 204

D. 404

POP QUIZ: REST

Which status code is best for a successful DELETE?

A. 200

B. 201

C. 204

D. 404

WHY APIS NEED AUTHENTICATION

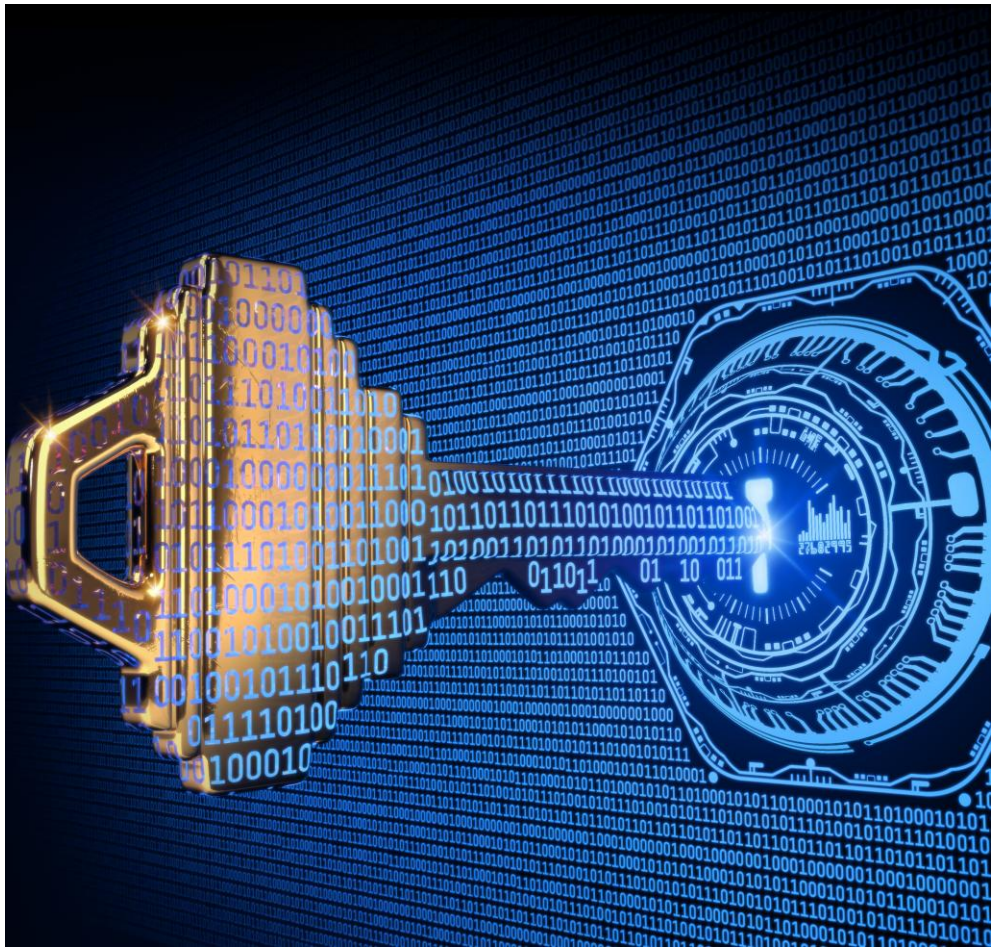
Most real-world APIs are not publicly accessible and require authentication. Authentication allows servers to identify who is making a request.

- Prevents unauthorized access
- Enables per-user permissions
- Allows usage tracking and auditing

WHAT IS AN API KEY?

An API key is a long, random value used to authenticate API requests. It is sent by the client with each request to prove identity.

- Acts like a password for programs
- Usually generated by the server
- Must be kept secret



API Keys in HTTP Headers

API keys are sent in HTTP headers to authenticate requests.

1. Client sends request with API key in header

Client

HTTP Request

```
GET /v1/data HTTP/1.1
Host: api.example.com
Authorization:  Bearer abc123xyz789
Accept: application/json
...
```

Common header formats

- `Authorization: Bearer <token>`
- `X-API-Key: <token>`

Server

2. Server reads and validates the API key

Example: Reading in Python (Flask)

```python
from flask import request, jsonify

@app.route('/v1/data')
def get_data():
    api_key = request.headers.get('Authorization', '')
    # Expecting: Bearer <token>
    if not api_key.slartswitk('Bearer '):
        return jsonify({'error': 'Missing or invalid API key'}), 401

    token = api_key.split(' ', 1)[1]  # Extract the token

    # Validate the token (lookup in DB, cache, etc.)
    if not is_valid_token(token):
        return jsonify({'error': 'Invalid API key'}), 401

    # Authorized - process the request
    return jsonify({'data': 'Here is your data'})
```

What server does

- Reads the API key from request headers
- Validates the key
- Allows access if valid
- Rejects with 401 if missing or invalid

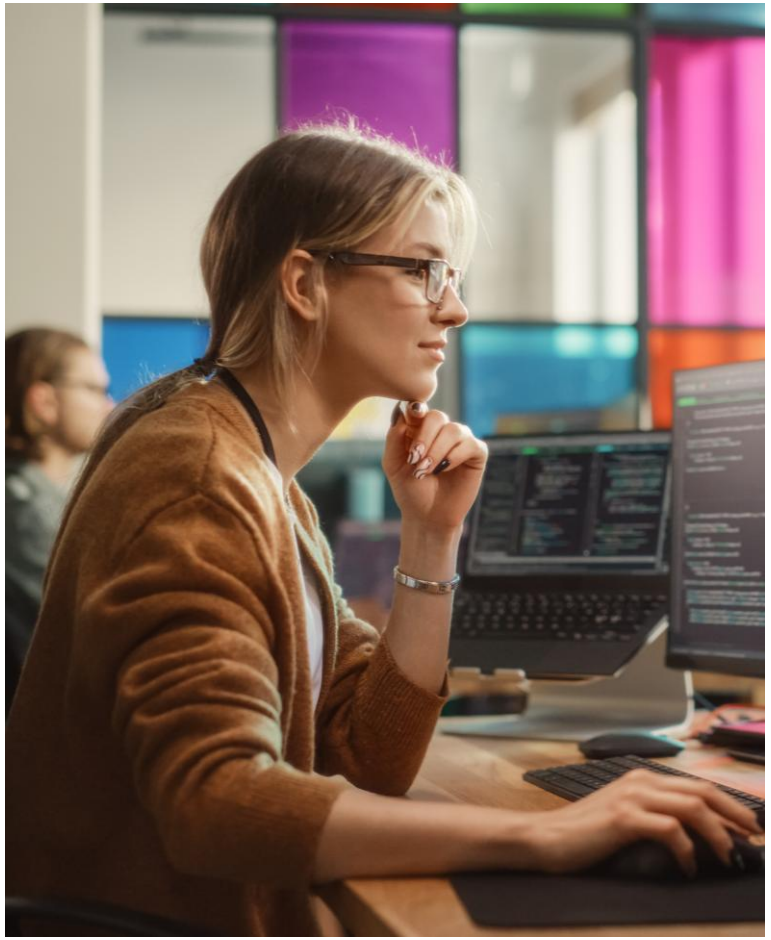
Why headers?

Headers keep API keys out of URLs, avoiding exposure in logs, browser history, and caches.

API KEYS IN THE REAL WORLD

Most cloud and SaaS platforms rely on API keys for automation access. Keys are designed for scripts, CI/CD, and system-to-system communication.

- Used by AWS, GitHub, Docker, and others
- Common in CLI tools and pipelines
- Often permission-scoped
- Most often you log into a website, generate the key, keep it secret and use it to access the API



SECURITY EXPECTATIONS

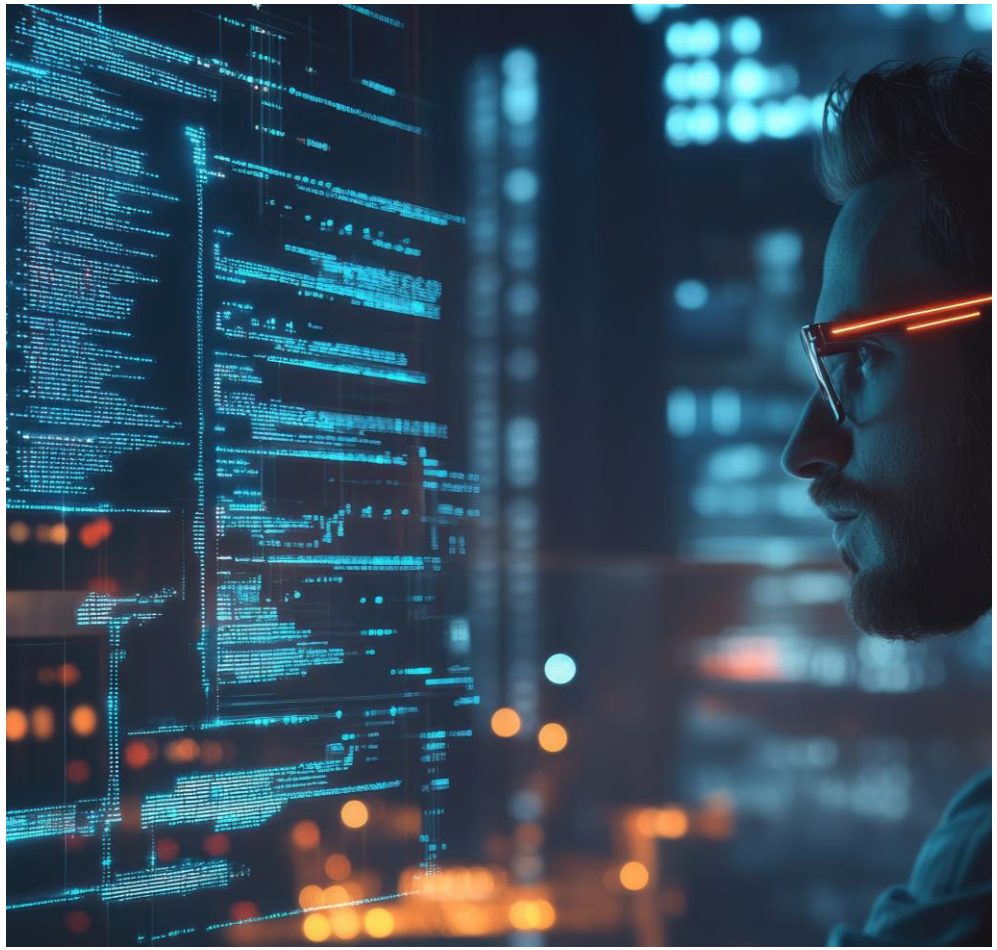
API keys must be treated with the same care as passwords. Poor key handling leads directly to security breaches.

- Never commit keys to source control
- Rotate keys regularly
- Revoke compromised keys immediately

CUSTOM HEADERS VS AUTHORIZATION

APIs commonly accept keys using the Authorization header or a custom header. Both approaches work, but Authorization is more standardized.

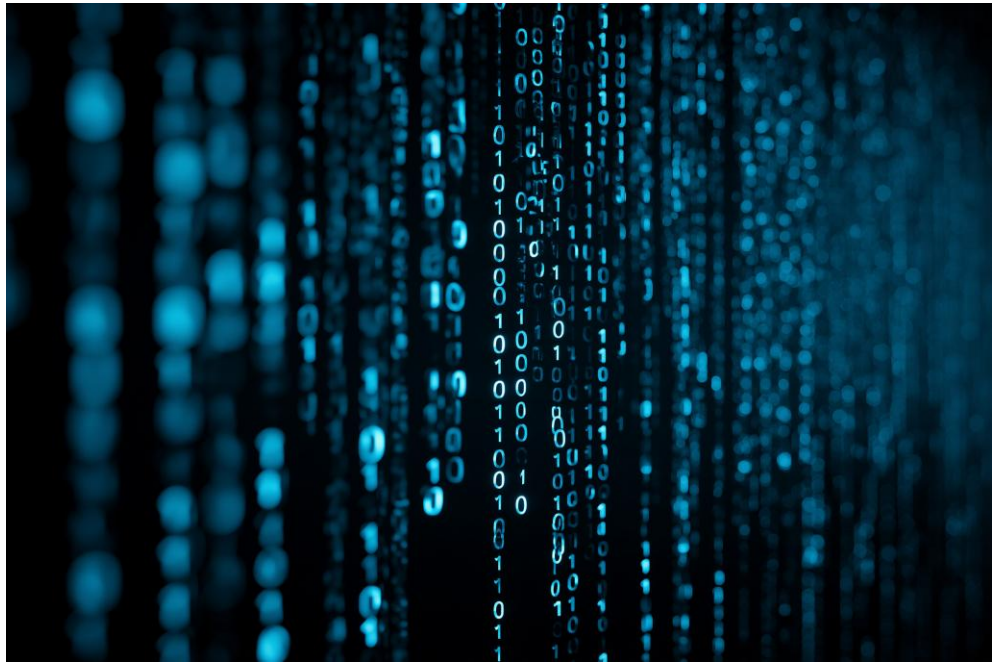
- Authorization: Bearer <key>
- X-API-Key: <key>
- Be consistent across endpoints



RANDOMNESS MATTERS

API keys must be difficult to guess. Predictable keys are vulnerable to brute-force attacks.

- Use secrets module
- Avoid sequential or short keys
- Prefer long random values
- Don’t store raw keys in Database. Store Hash of key



WHAT API KEYS DO

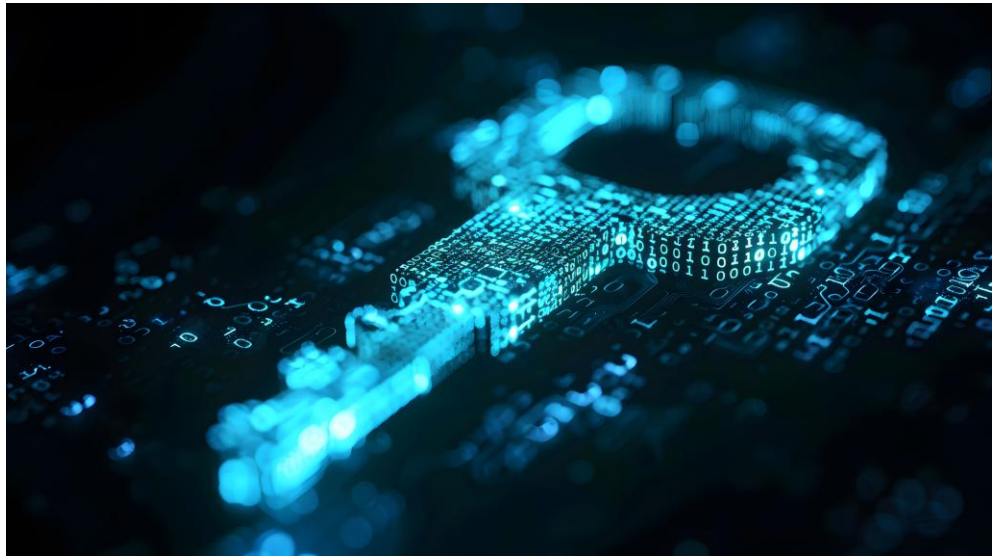
API keys identify the caller and allow access decisions. They enable rate limits, permissions, and auditing.

- Identify who is calling
- Control access scope
- Support monitoring

WHAT API KEYS DO NOT DO

API keys do not encrypt network traffic. They rely on HTTPS for confidentiality.

- No encryption by themselves
- Visible on the wire without HTTPS
- Must be combined with TLS



LAB 2.1

HTTPS IS STILL REQUIRED

Without HTTPS, API keys can be intercepted in transit. Encryption is mandatory for any authenticated API.

- Protects against sniffing
- Prevents man-in-the-middle attacks
- Required for secure cookies

WHY HTTP IS DANGEROUS

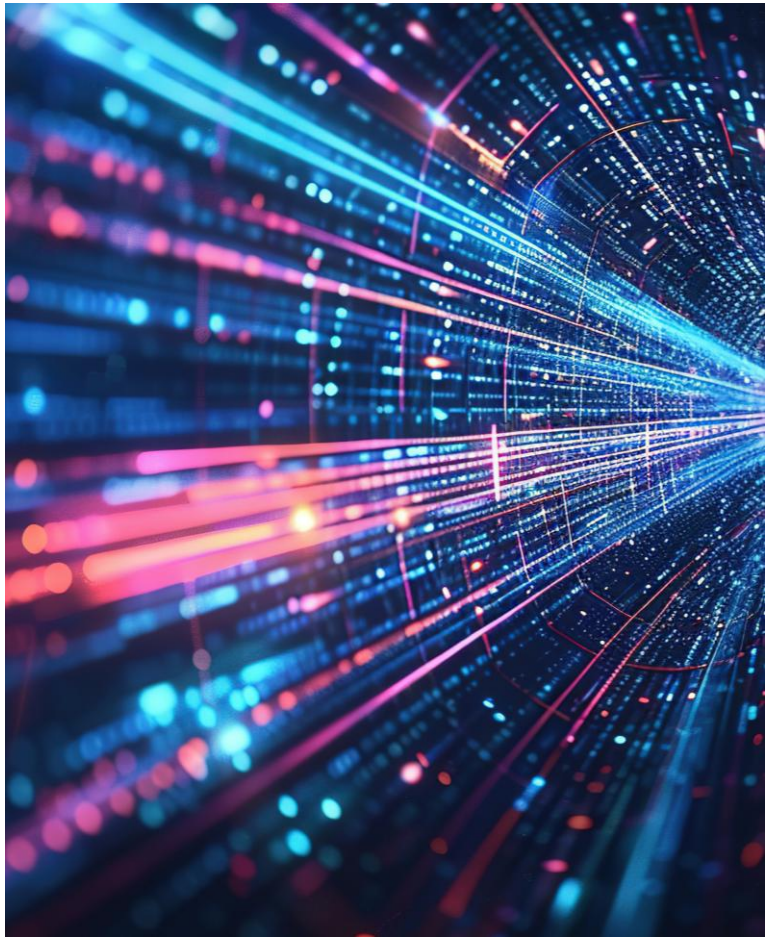
Plain HTTP sends data in cleartext. Anyone on the network can intercept credentials.

- Passwords exposed
- Tokens leaked
- Man-in-the-middle attacks

TLS IN SIMPLE TERMS

TLS creates a secure tunnel before any data is exchanged. All application data flows through this encrypted channel.

- Handshake
- Key exchange
- Secure session



PUBLIC AND PRIVATE KEYS

TLS relies on asymmetric cryptography. Public keys are shared while private keys remain secret.

- Public key advertised
- Private key protected
- Trust model

CERTIFICATES

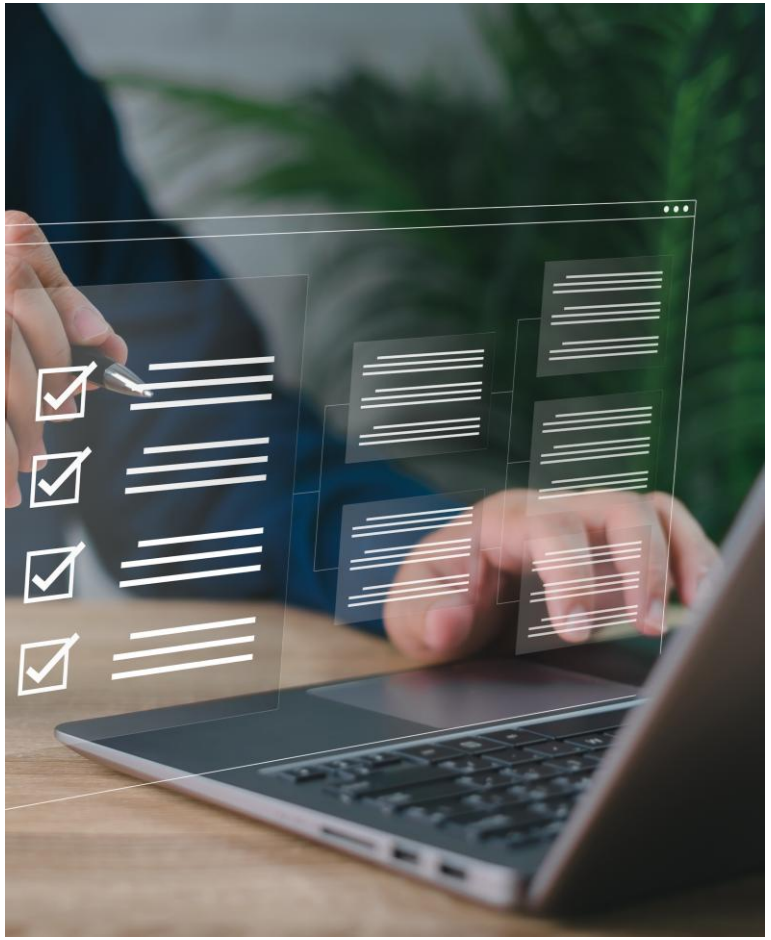
Certificates bind a public key to a server identity. Clients trust certificates signed by known authorities.

- Contains public key
- Signed by CA
- Proves identity

DIGITAL SIGNATURES

Servers prove ownership of private keys during TLS handshakes. Clients verify signatures using public keys.

- Authentication
- Integrity
- Non-repudiation



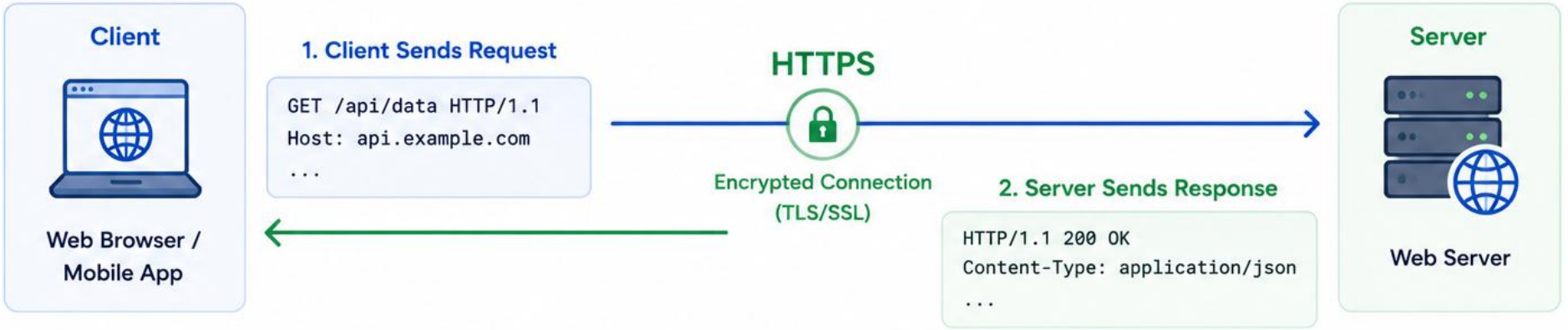
ENCRYPTION AFTER HANDSHAKE

Once TLS is established, symmetric encryption is used. This keeps communication fast and secure.

- Session keys
- Efficient encryption
- Secure channel

HTTPS (HyperText Transfer Protocol Secure)

HTTPS is HTTP over TLS/SSL. It encrypts data between the client and server to provide **confidentiality**, **integrity**, and **authentication**.



How HTTPS Works (Simplified)

1. TLS Handshake
Client and server negotiate TLS version, verify certificates, and establish a secure session.

2. Encrypted Data Transfer
All data (requests and responses) is encrypted during transmission.

3. Decryption
The server decrypts the request to process it. The client decrypts the response.

4. Secure & Trusted
Data remains private and tamper-proof while in transit.

Why HTTPS?

Encryption
Protects data from eavesdropping.

Integrity
Ensures data is not tampered with.

Authentication
Verifies the server's identity using certificates.

Better Trust & SEO
Browsers show a padlock. Search engines prefer HTTPS.

Key Point: Always use HTTPS to protect users and build trust.

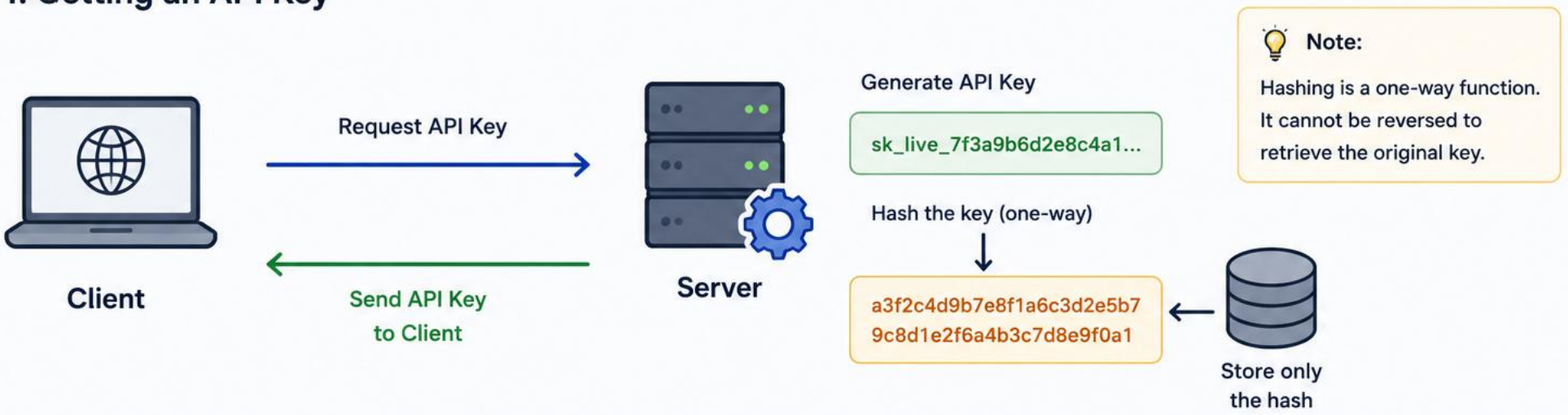
https://api.example.com

WHY HASH API KEYS

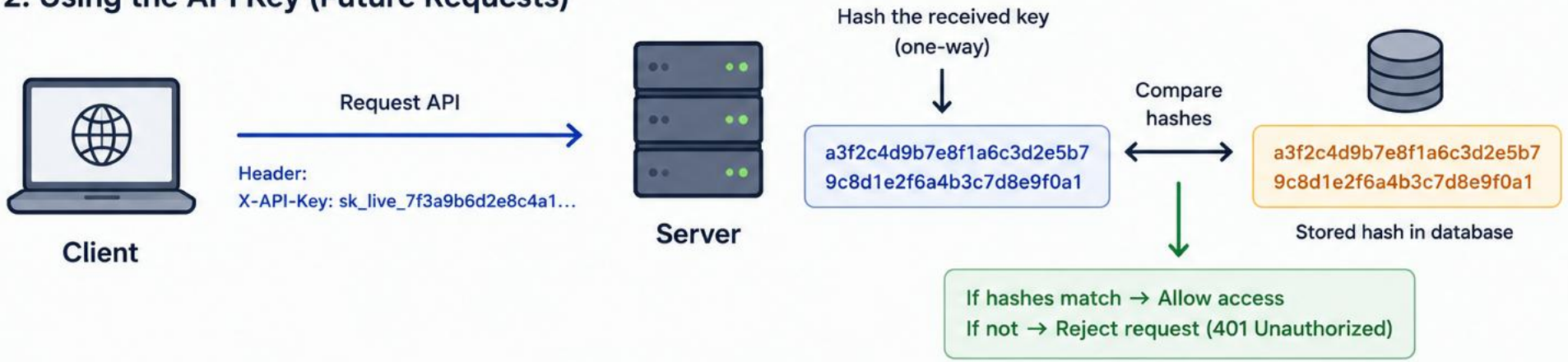
Storing raw API keys creates a single point of failure. Hashing ensures leaked databases cannot be used directly for access.

- Protects against database leaks
- Industry best practice
- Same principle as password storage

1. Getting an API Key



2. Using the API Key (Future Requests)



Why hash? If the database is leaked, attackers only get the hash, not the actual API keys. Hashing protects your users.

WHY PYTHON HASH() IS NOT SECURE

Python's built-in hash() is designed for internal hash tables (dict), not security. Its output is intentionally unstable across runs.

- Changes between executions
- Not cryptographically secure
- Never store security data with it
- Use alternatives like bcrypt, scrypt, or Argon2

WHAT IS HMAC

HMAC combines a hash function with a server-side secret. This prevents offline guessing even if hashes are leaked.

- Uses a shared secret
- Secret is not stored in database
- Stronger than plain hashing

AUTHENTICATION VS AUTHORIZATION

Authentication identifies who is calling the API. Authorization determines what they are allowed to do.

- Key validates identity
- Role controls access
- Separate concerns

POP QUIZ: REST

Which header is commonly used to send API keys?

A. Cookie

B. Authorization

C. Content-Type

D. Host

POP QUIZ: REST

Which header is commonly used to send API keys?

A. Cookie

B. Authorization

C. Content-Type

D. Host

POP QUIZ: REST

Why should API keys not be placed in URLs?

A. URLs cannot carry headers

B. URLs are slower

C. URLs are logged and cached

D. Browsers block them

POP QUIZ: REST

Why should API keys not be placed in URLs?

A. URLs cannot carry headers

B. URLs are slower

C. URLs are logged and cached

D. Browsers block them

POP QUIZ: REST

What is the main purpose of hashing API keys?

A. Compress keys

B. Encrypt traffic

C. Protect against database leaks

D. Improve performance

POP QUIZ: REST

What is the main purpose of hashing API keys?

A. Compress keys

B. Encrypt traffic

C. Protect against database leaks

D. Improve performance

POP QUIZ: REST

Which status code should be returned for a missing API key?

A. 400

B. 401

C. 403

D. 404

POP QUIZ: REST

Which status code should be returned for a missing API key?

A. 400

B. 401

C. 403

D. 404

WHAT IS AUTHENTICATION STATE

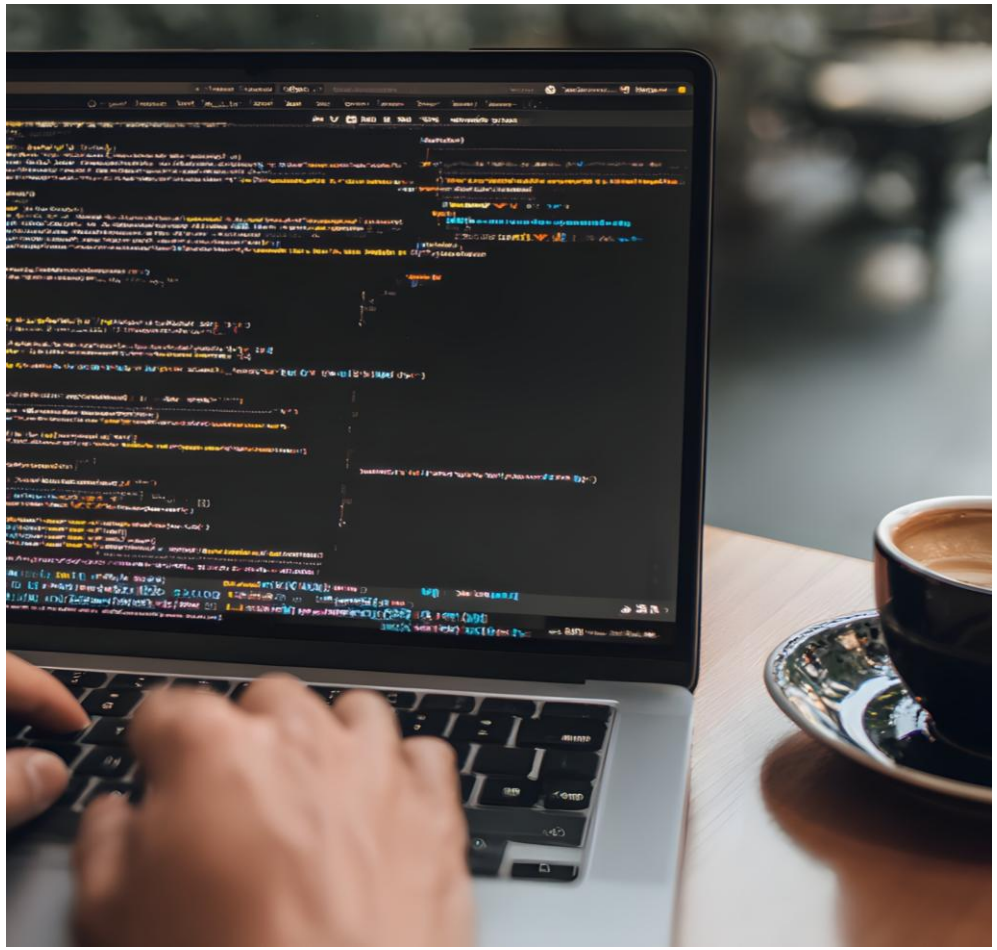
Authentication state tracks who a user is across requests. Different systems store this state in different places.

- Session IDs
- Cookies
- Tokens

SESSION-BASED AUTHENTICATION

Traditional web apps rely on server-managed session identifiers. The server must remember every active user session.

- Random session ID
- Stored server-side
- Mapped to user



HOW SESSIONS WORK

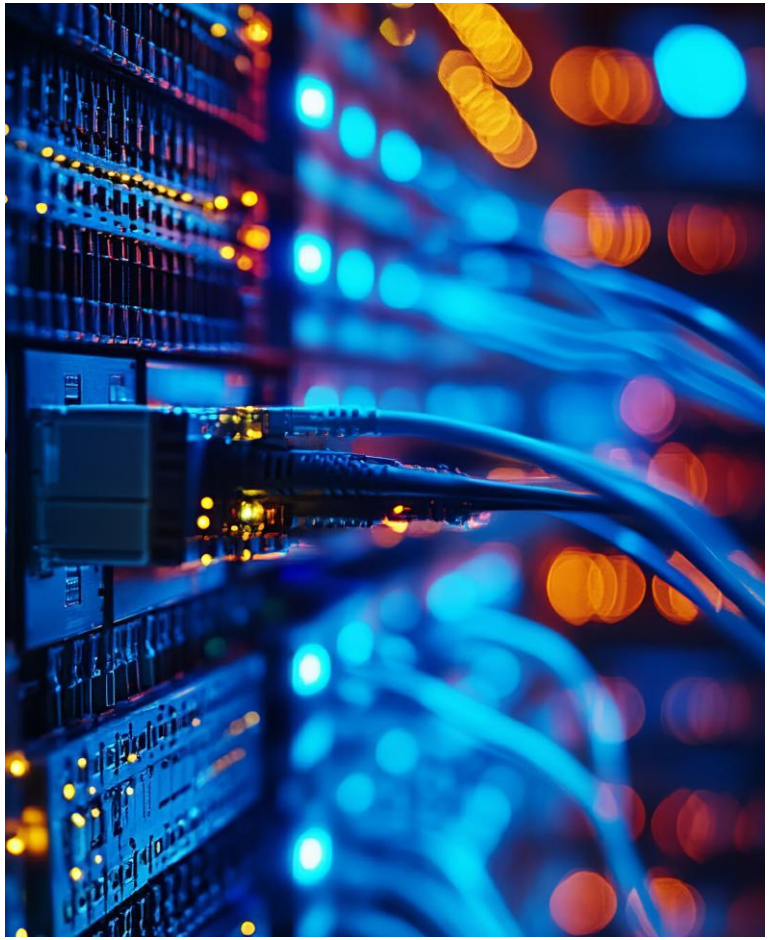
After login, the server creates a session and sends the ID to the client. Every request must be validated against session storage.

- Session ID in cookie
- Lookup on each request
- Stateful design

SESSION STORAGE OPTIONS

Sessions can be stored in memory or external systems. Scaling requires shared session storage.

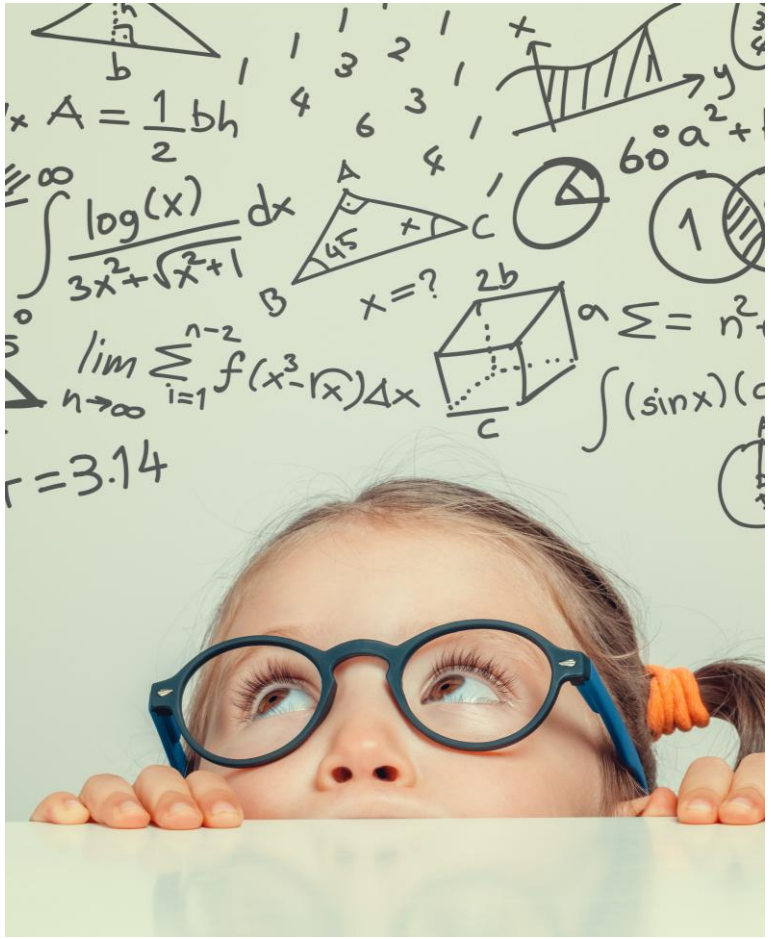
- In-memory
- Redis
- Databases



PROBLEMS WITH SESSIONS

Sessions increase server complexity and coupling.
Scaling horizontally becomes harder.

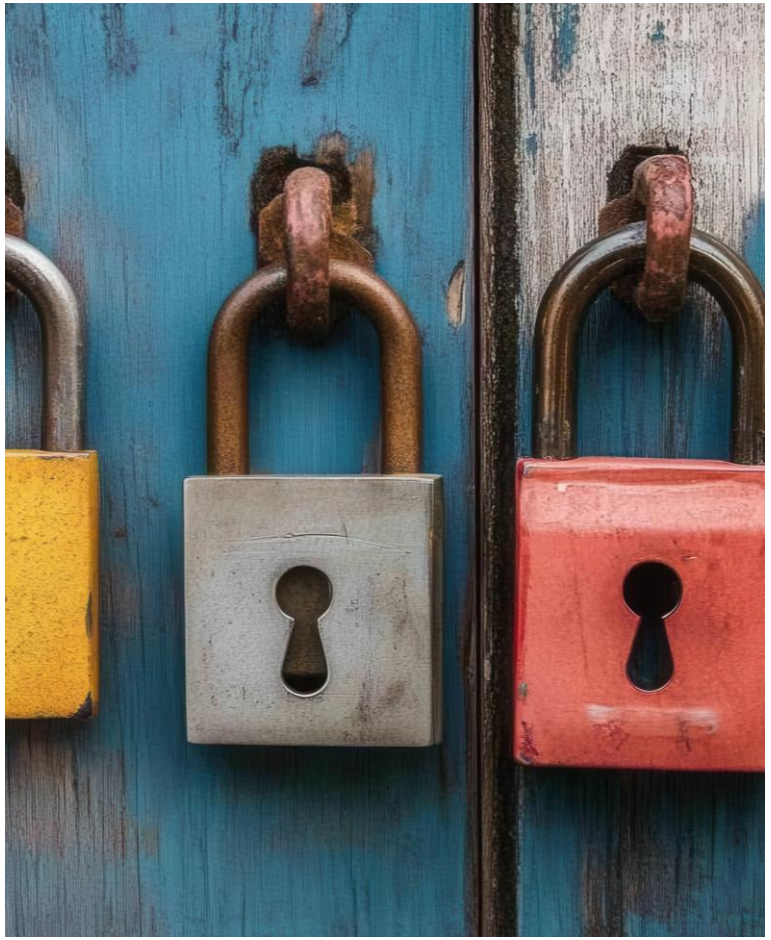
- Sticky sessions
- Extra infrastructure
- Operational overhead



INTRODUCING JWT

JSON Web Tokens embed authentication claims inside the token itself. The server does not store per-user session state.

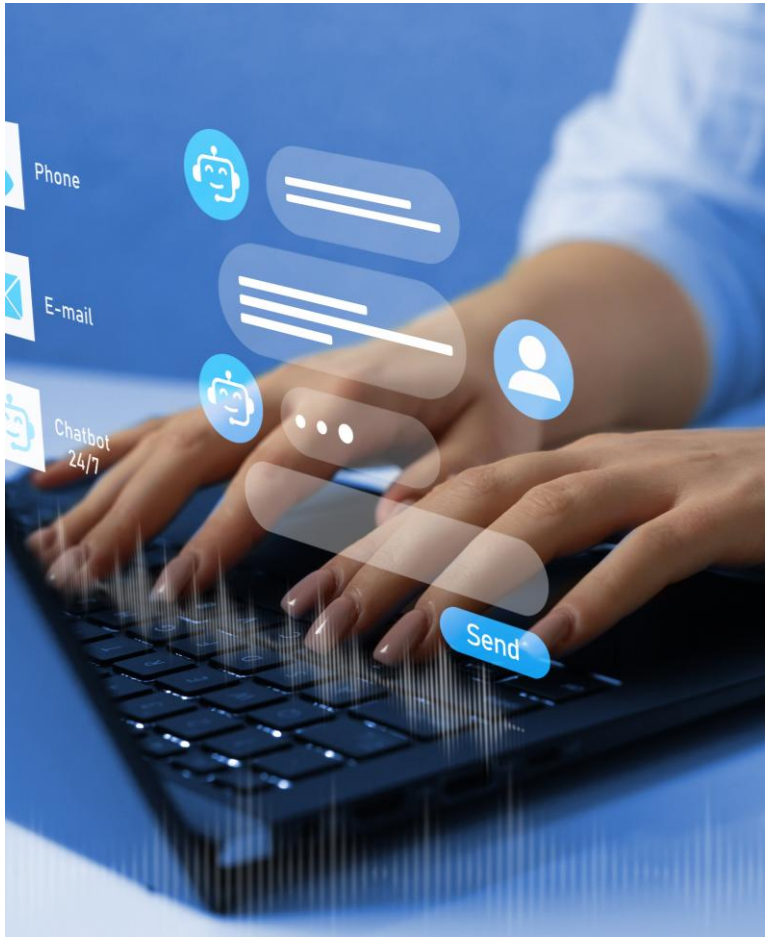
- Self-contained token
- Signed not encrypted
- Stateless



WHAT JWT CONTAINS

JWTs carry claims about the user and expiration time. Anyone can read the payload, but only the server can sign it and verify that it came from itself.

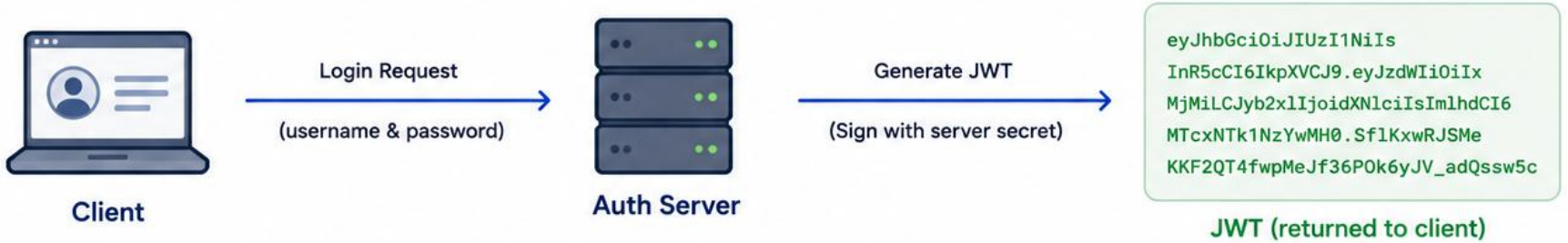
- Username
- Expiration
- Signature



JSON Web Token (JWT)

A compact, URL-safe token used to securely transmit information between parties.

1. Authentication Flow (Obtain Token)



What's Inside a JWT?

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjMiLCJyb2xlIjoidXNlciIsImlhdCI6MTcxNTk1NzYwMH0.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Header (Algorithm & Token Type)

Payload (Claims / Data)

Signature (Verify Integrity)

Example Payload (Decoded)

```
{
 "sub": "123",
 "role": "user",
 "iat": 1715957600
}
```

JWTs are signed, not encrypted. Anyone can **read** the payload, but cannot tamper with it.

The signature ensures the token came from the server and has not been modified.

2. Using the JWT (Future Requests)

1. Verify signature using server secret/public key
2. Validate claims (exp, issuer, etc.)
3. If valid → Allow access
   If invalid → Reject (401 Unauthorized)

Benefits of JWTs

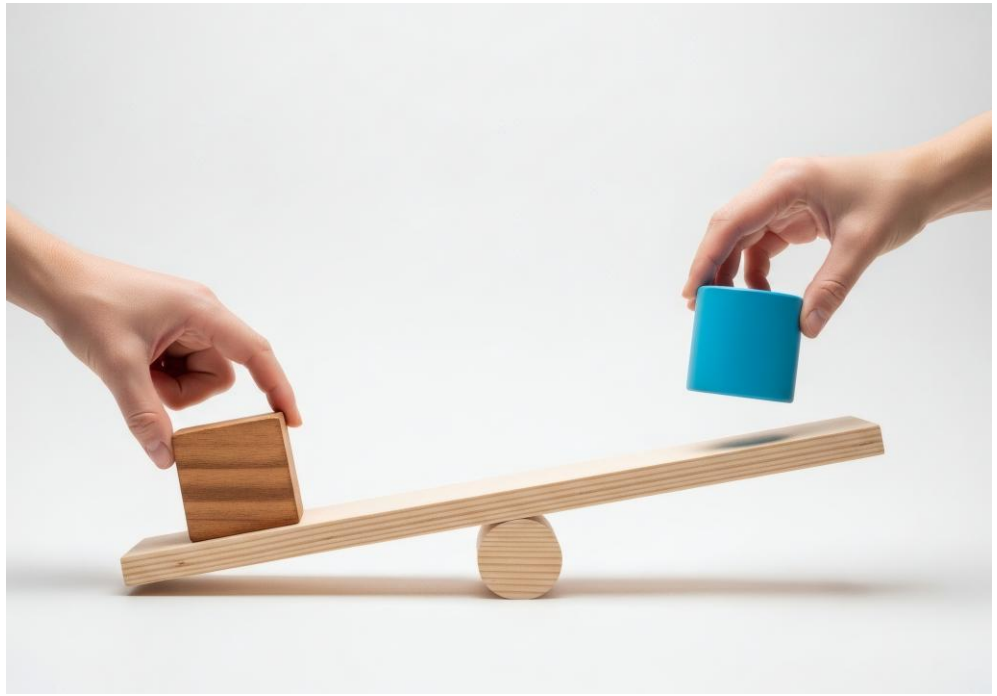
- **Stateless:** No server-side session storage
- **Scalable:** Works across multiple services
- **Self-contained:** Includes claims (user info, roles, etc.)
- **Secure:** Signed to prevent tampering

Key Point: Keep tokens safe! Store securely on the client (e.g., httpOnly cookies or secure storage) and always use HTTPS when transmitting tokens.

JWT VS SESSIONS SUMMARY

JWTs trade revocation simplicity for scalability. Sessions trade scalability for control.

- Stateless vs stateful
- Scale vs revoke
- Choose by use case



From Login to Using an API Key

JWT for authentication, API key for programmatic access

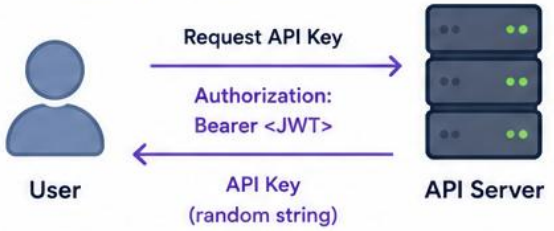
1 User logs in

User sends username and password to log in.

2 User receives JWT

Auth server validates credentials and returns a JWT.

3 User requests an API Key using the JWT

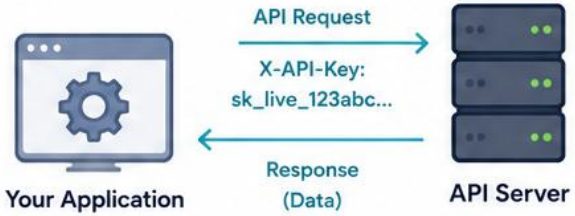


User sends the JWT to the API server and receives a new API key.

4 Store API Key as a secret for your application

User stores the API key securely (e.g., .env file, secret manager).

5 Application uses API Key



Your application includes the API key in requests to access the API.

6 API Server validates the API Key

- Receives API key
- Validates key
- If valid → allow access
  If invalid → deny access

The API server validates the API key and processes the request.

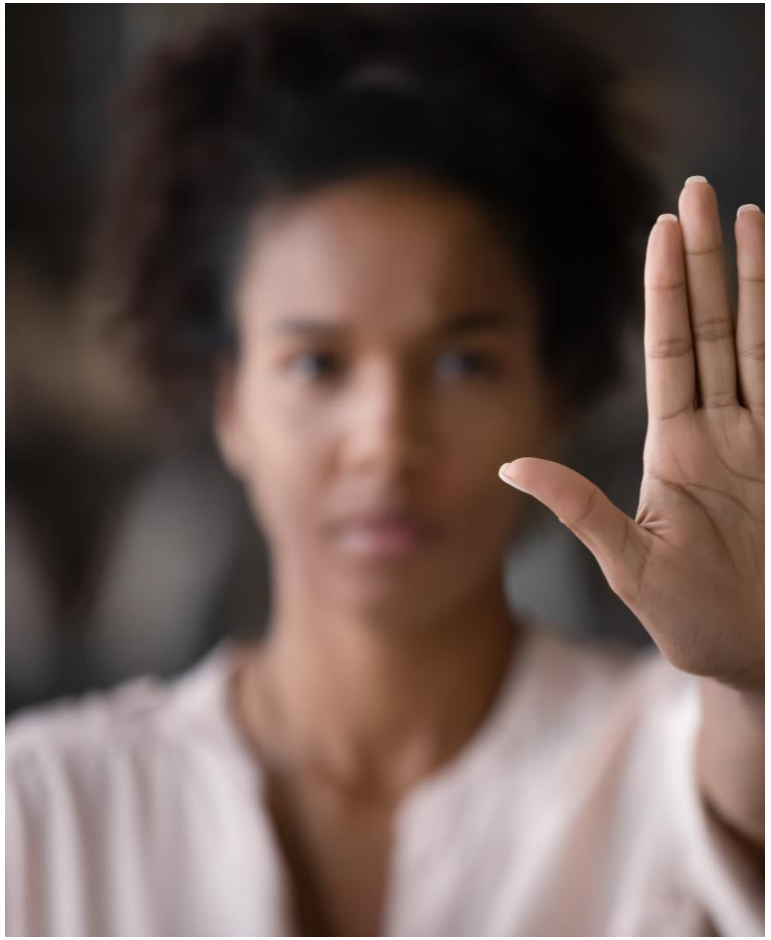
Key Concepts

- JWT: Used for user authentication (short-lived, proves identity)
- API Key: Used by applications for programmatic access (longer-lived)
- Keep your API key secret and never expose it in client-side code

JWT EXPIRATION

JWTs always include an expiration timestamp.
Expired tokens must be rejected.

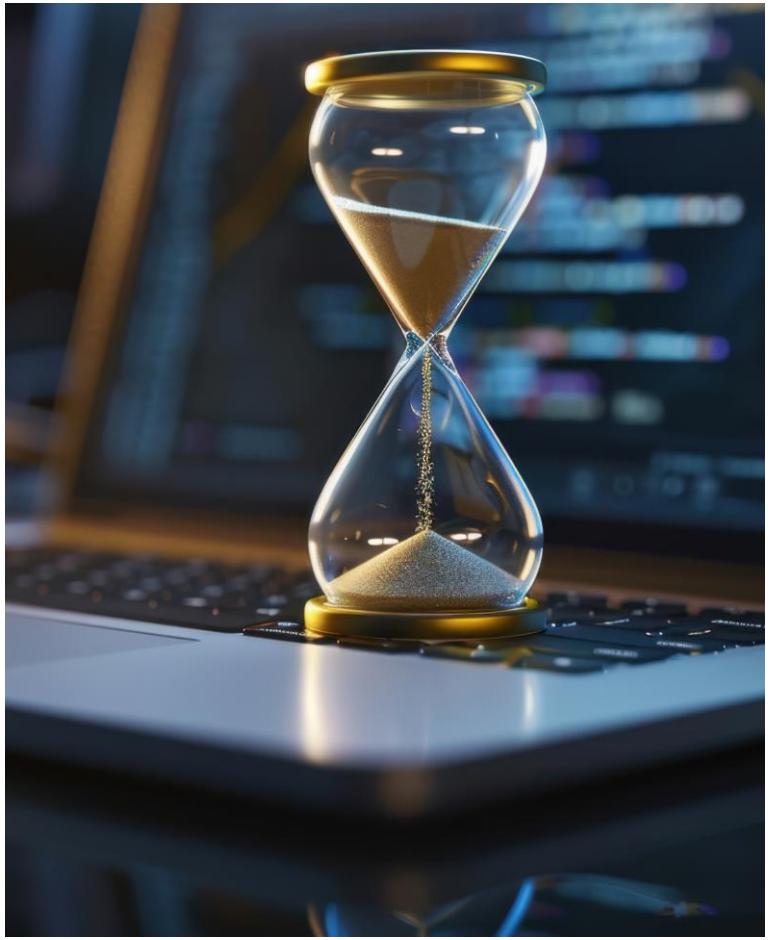
- Limits damage if stolen
- Encourages re-authentication
- Time-based control



WHY SHORT-LIVED TOKENS

JWTs cannot be instantly revoked. Short lifetimes reduce risk.

- Minutes not days
- Rotate often
- Delete cookie on logout



JWT TRANSPORT OPTIONS

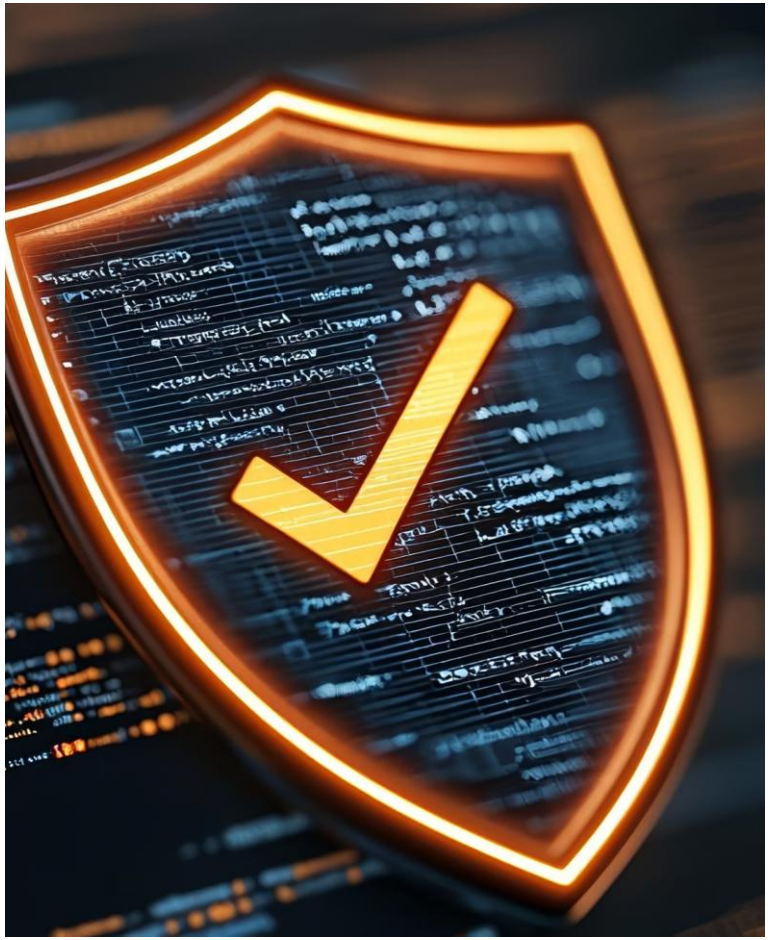
JWTs can be sent via headers or cookies. Cookies are preferred for browsers.

- Authorization header
- HttpOnly cookie
- SameSite support

HTTPONLY COOKIES

HttpOnly cookies cannot be accessed by JavaScript. This protects tokens from XSS.

- Browser-managed
- Not readable via JS
- Safer default



SAMESITE COOKIES

SameSite controls cross-site cookie behavior. It reduces CSRF risk.

- Strict mode
- Lax mode
- None

SECURE COOKIE FLAG

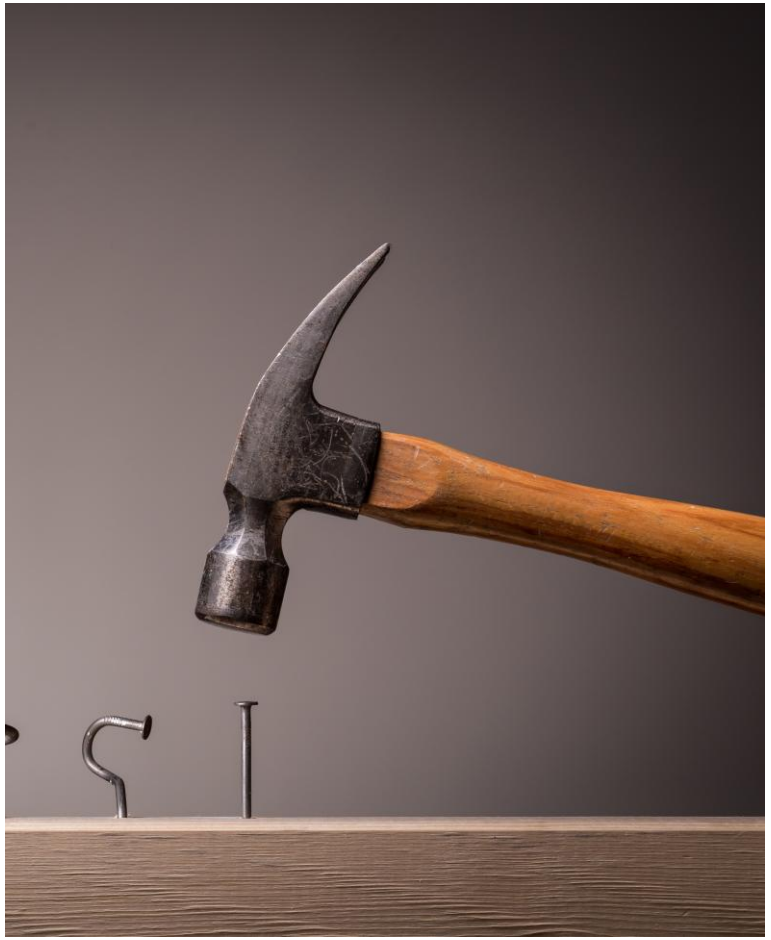
Secure cookies are only sent over HTTPS. They prevent token leakage on plaintext connections.

- Requires HTTPS
- Protects credentials
- Mandatory in production

JWT VALIDATION ERRORS

JWT verification can fail for multiple reasons. Each failure must be handled safely.

- Missing token
- Expired token
- Invalid signature



JWT LOGOUT BEHAVIOR

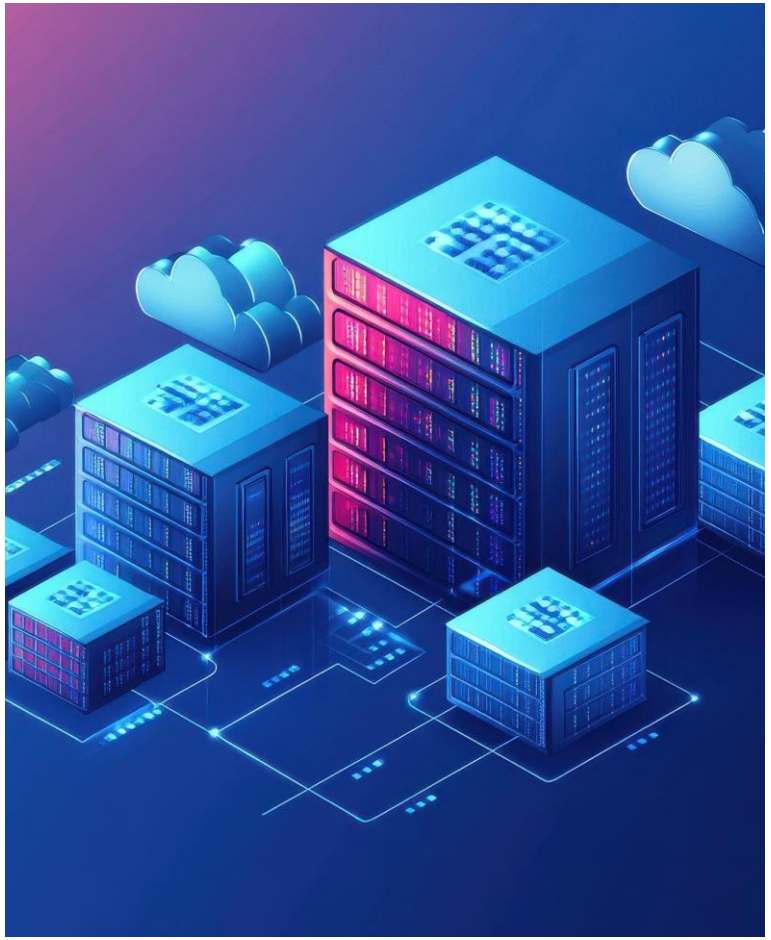
Logging out deletes the JWT cookie. The token itself remains valid until expiration.

- Client-side removal
- No server revocation
- Expiration enforced

STATELESS SCALING BENEFITS

JWTs allow horizontal scaling without shared session stores. Any server can validate any token.

- No Redis needed
- No sticky sessions
- Simpler infrastructure



LAB 2.2

LAB 2.3

POP QUIZ: REST

What makes JWT authentication stateless?

A. Cookies store sessions

B. Tokens store claims

C. Server stores sessions

D. Redis is optional

POP QUIZ: REST

What makes JWT authentication stateless?

A. Cookies store sessions

B. Tokens store claims

C. Server stores sessions

D. Redis is optional

POP QUIZ: REST

Why are JWTs usually short-lived?

A. They are encrypted

B. They are cached

C. They cannot be revoked

D. They are compressed

POP QUIZ: REST

Why are JWTs usually short-lived?

A. They are encrypted

B. They are cached

C. They cannot be revoked

D. They are compressed

POP QUIZ: REST

Which cookie flag prevents JavaScript access?

A. Secure

B. SameSite

C. HttpOnly

D. Path

POP QUIZ: REST

Which cookie flag prevents JavaScript access?

A. Secure

B. SameSite

C. HttpOnly

D. Path

POP QUIZ: REST

What happens when a JWT signing secret is rotated?

A. Tokens are refreshed

B. Old tokens become invalid

C. Cookies persist

D. Sessions migrate

POP QUIZ: REST

What happens when a JWT signing secret is rotated?

A. Tokens are refreshed

B. Old tokens become invalid

C. Cookies persist

D. Sessions migrate

POP QUIZ: REST

Which risk still exists when using JWT cookies?

A. SQL injection

B. XSS

C. CSRF

D. Brute force

POP QUIZ: REST

Which risk still exists when using JWT cookies?

A. SQL injection

B. XSS

C. CSRF

D. Brute force

POP QUIZ: REST

What does HTTPS primarily protect?

A. Application logic

B. Data in transit

C. Server uptime

D. Database encryption

POP QUIZ: REST

What does HTTPS primarily protect?

A. Application logic

B. Data in transit

C. Server uptime

D. Database encryption

CONCLUSION

Today you learned the core concepts that power modern web APIs. Understanding these fundamentals provides the foundation needed to design, consume, secure, and troubleshoot APIs in real-world applications.

Most modern applications communicate through APIs. Mastering these fundamentals is the first step toward building and integrating secure, scalable systems